

Note 3: Deploying to Servers and Cloud Using GitHub Actions

Deployment Patterns

What Happens After the Build?

Once a Docker image is built and pushed to a registry, it needs to be deployed somewhere — a cloud server, a VM, a Kubernetes cluster, or a managed cloud service. GitHub Actions can automate this final step.

Deploying to a Remote Server via SSH

The most straightforward deployment approach — SSH into a server and run commands to pull the latest image and restart the container.

- name: Deploy to server via SSH

uses: appleboy/ssh-action@v1.0.3

with:

host: \${{ secrets.SERVER_HOST }}

username: \${{ secrets.SERVER_USER }}

key: \${{ secrets.SERVER_SSH_KEY }}

script: |

docker pull ghcr.io/myusername/my-app:latest

docker stop my-app || true

docker rm my-app || true

docker run -d \

--name my-app \

-p 8080:8080 \

--restart unless-stopped \

ghcr.io/myusername/my-app:latest

Required secrets:

- SERVER_HOST — IP address or domain of the server
- SERVER_USER — SSH username (e.g., ubuntu, ec2-user)
- SERVER_SSH_KEY — private SSH key for authentication

The `|| true` after `docker stop` and `docker rm` prevents the script from failing if the container does not exist yet (first deployment).

Deploying to AWS EC2 / ECS

EC2 (SSH-based, similar to above)

For EC2 instances, the same SSH approach works. Add the EC2 instance's public IP as `SERVER_HOST` and the EC2 key pair's private key as `SERVER_SSH_KEY`.

ECS (Elastic Container Service)

AWS ECS is a managed container orchestration service. Deploying involves updating the ECS task definition with the new image and triggering a service update:

- name: Configure AWS credentials

uses: aws-actions/configure-aws-credentials@v4

with:

aws-access-key-id: \${{ secrets.AWS_ACCESS_KEY_ID }}

aws-secret-access-key: \${{ secrets.AWS_SECRET_ACCESS_KEY }}

aws-region: us-east-1

- name: Log in to Amazon ECR

id: login-ecr

uses: aws-actions/amazon-ecr-login@v2

- name: Deploy to ECS

uses: aws-actions/amazon-ecs-deploy-task-definition@v1

with:

task-definition: task-definition.json

service: my-app-service

cluster: my-cluster

wait-for-service-stability: true

Environment Protection Rules

For production deployments, GitHub supports environments with protection rules — requiring manual approval before a deployment job runs.

jobs:

deploy-production:

runs-on: ubuntu-latest

environment:

name: production

url: https://myapp.com

needs: [build, test]

steps:

- name: Deploy to production

run: ./deploy-prod.sh

In the GitHub repository settings under Environments, you can configure:

- Required reviewers — specific people who must approve before the job runs
- Wait timer — a delay before the deployment starts
- Branch restrictions — only allow deployments from specific branches

When the workflow reaches the deploy-production job, it pauses and sends a notification to the required reviewers. The job only proceeds after approval.

Using GitHub Actions Secrets for Deployments

All sensitive values used in deployments — SSH keys, API tokens, cloud credentials — must be stored as GitHub secrets, never hardcoded in workflow files.

Secret Type	Where to Store	Scope
Repository secrets	Settings → Secrets → Actions	Available to all workflows in the repo
Environment secrets	Settings → Environments → Secrets	Only available to jobs targeting that environment
Organisation secrets	Organisation Settings → Secrets	Available across multiple repositories

Accessing secrets in workflows:

env:

```
API_KEY: ${{ secrets.MY_API_KEY }}
```

Or directly in a step:

```
- run: ./deploy.sh
```

env:

```
TOKEN: ${{ secrets.DEPLOY_TOKEN }}
```

Secrets are masked in workflow logs — if a secret value appears in any log output, GitHub automatically replaces it with ***.

Complete CI/CD Workflow Summary

Stage	Trigger	Actions Performed
-------	---------	-------------------

CI on PR	pull_request to main	Checkout, build, test, code quality checks
----------	----------------------	--

CI on merge	push to main	Build, test, build Docker image, push to GHCR
-------------	--------------	---

CD to staging	push to main	SSH deploy or cloud deploy to staging
---------------	--------------	---------------------------------------

CD to production	Manual approval	Deploy to production after reviewer approval
------------------	-----------------	--